1 محمد مصطفى محمد المكاوي

قائد الفريق · الـ Core و FCFS

📌 شو إلك بالمشروع

المادة نظم تشغيل: ابدأ دايماً بشرح الفكرة (المربعات البنفسجية 🧠) وبعدين ورجي كيف طبّقناها بالكود.

- **Phase 2 · فيديو 19 نوفمبر:** Process input، الـ PCB model، و FCFS
- **Phase 3 · 17 ديسمبر:** الـ Random workload وتجميع كل الأجزاء

الكود كله بملف **CPU_SchedLab.py**. افتحه بـ VS Code واكبس **Ctrl + G** واكتب رقم السطر اللي بالصورة.

Phase 2 · فيديو 19 نوفمبر — Process input، الـ PCB model، و FCFS

🎤 البداية — التعريف بالفريق والمشروع

السلام عليكم دكتورة، أنا محمد مصطفى محمد المكاوي، قائد فريق **Kernel Crew**. مشروعنا اسمه **CPU SchedLab**، وهو برنامج بيوضّح كيف نظام التشغيل بيقرر مين الـ process اللي رح تشتغل بعدين على المعالج. بندخّل الـ processes، بنختار الخوارزمية، وبنشوف الـ Gantt chart وهو بينرسم، والبرنامج بيحسبلنا الأرقام لحاله. الفريق خمسة: أنا مسؤول عن الـ core وإدخال الـ processes و FCFS، سامي عن SJF و SRTF، محمد الدلاعين عن Round Robin والـ context switch والمقارنة، عروب عن Priority والمقاييس، وسارة عن الواجهة والـ Gantt chart. وهسا رح أشرحلك الجزء تبعي.

🧠 الفكرة بنظام التشغيل · شو يعني Process وشو الـ PCB؟

الـ **process** هو برنامج وهو شغّال (program in execution). نظام التشغيل بيحفظ لكل process سجل اسمه **PCB (Process Control Block)**، وفيه: حالة الـ process (state)، والـ program counter، والـ registers، ومعلومات الجدولة زي الـ priority، ومعلومات الذاكرة والـ I/O. الـ process بتمرق بحالات: **new** ← **ready** ← **running** ← **waiting** ← **terminated**. كل الـ processes اللي بحالة **ready** بتستنّى بطابور اسمه **ready queue** لحد ما ياخدوا المعالج. بمشروعنا، الكلاس Process بيمثّل الجزء من الـ PCB اللي بيهم الجدولة: الاسم، ووقت الوصول للـ ready queue، والـ burst، والـ priority.

🧠 الفكرة بنظام التشغيل · شو يعني CPU Scheduling؟

لما يكون عنا معالج واحد وكذا process جاهزة، لازم حدا يقرر مين يشتغل. هاد شغل الـ **short-term scheduler (CPU scheduler)**: بيختار process من الـ ready queue. بعدين الـ **dispatcher** بيعطيها المعالج فعلياً: بيعمل context switch، وبيرجع لـ user mode، وبينطّ لمكان ما وقفت. والجدولة نوعين: **non-preemptive**، يعني الـ process بتضل ماسكة المعالج لحد ما تخلص أو تروح تستنّى I/O، و**preemptive**، يعني النظام بيقدر ياخد منها المعالج بنص الشغل. وعشان نقارن بين الخوارزميات بنستخدم **scheduling criteria**: بدنا CPU utilization و throughput أعلى اشي، و turnaround و waiting و response time أقل اشي.

الـ Process (الـ PCB)

CPU_SchedLab.py lines 34–50

```python
@dataclass
class Process:
    """One process, like a small PCB. Smaller priority number = more important."""
    pid: str
    arrival: int
    burst: int
    priority: int = 0

    def __post_init__(self):
        if not str(self.pid).strip():
            raise ValueError("Process id cannot be empty.")
        if self.arrival < 0:
            raise ValueError(f"{self.pid}: arrival time cannot be negative.")
        if self.burst <= 0:
            raise ValueError(f"{self.pid}: burst time must be greater than 0.")
        if self.priority < 0:
            raise ValueError(f"{self.pid}: priority cannot be negative.")
```

🎤 شو بتحكي للدكتورة

هون يا دكتورة عرّفت كلاس اسمه **Process**، وهاد بيمثّل الـ process جوّا نظام التشغيل، يعني زي نسخة مصغّرة من الـ **PCB**. كل process إلها أربع معلومات: اسمها **pid**، ووقت وصولها **arrival**، وقدّيش بدها وقت على المعالج وهو الـ **burst**، والـ **priority** تبعتها، وكل ما كان رقم الـ priority أصغر كل ما كانت أهم، زي الكتاب بالزبط. وبالدالة **__post_init__** بفحص البيانات أول ما تنعمل الـ process: ما بسمح يكون الاسم فاضي، ولا وقت الوصول بالسالب، ولا الـ burst صفر. وإذا في غلط، بتطلع للمستخدم رسالة واضحة بتحكيله وين الغلط.

إدخال الـ Processes (F1)

CPU_SchedLab.py lines 83–91

```python
def make_process(pid, arrival, burst, priority="0"):
    """Turn the text typed by the user into a Process, with a clear error message."""
    numbers = []
    for name, text in (("Arrival", arrival), ("Burst", burst), ("Priority", priority)):
        text = str(text).strip()
        if not text.lstrip("-").isdigit():
            raise ValueError(f"{name} must be a whole number (you typed '{text}').")
        numbers.append(int(text))
    return Process(str(pid).strip(), *numbers)
```

🎤 شو بتحكي للدكتورة

هاي الدالة **make_process** هي اللي بتاخد الأرقام اللي بيكتبها المستخدم بالواجهة وبتحوّلها لـ process. بتمرق على كل قيمة، وإذا المستخدم كتب حرف بدل رقم مثلاً، بتطلعله رسالة بتحكيله بالزبط شو اللي كتبه غلط. وهيك بنضمن إنه ما يفوت عالبرنامج اشي غلط.

🧠 الفكرة بنظام التشغيل · خوارزمية FCFS بالمادة

First-Come, First-Served هي أبسط خوارزمية: الـ ready queue بيمشي **FIFO**، اللي وصل أول بياخد المعالج أول، وهي **non-preemptive**. سهلة كتير، بس معدّل الانتظار فيها غالباً عالي. ومشكلتها المشهورة: **Convoy effect** process طويلة (زي process بتعتمد عالـ CPU) بتمسك المعالج، وكل الـ processes القصيرة بتتكدّس وراها زي السيارات ورا شاحنة. بمثالنا P1 طويلة وإجت أول، فـ P3 اللي بدها وحدة وحدة استنّت 6.

Process	Arrival	Burst	Completion	Turnaround	Waiting
P1	0	5	5	5	0
P2	1	3	8	7	4
P3	2	1	9	7	6



🧮 Waiting = Turnaround − Burst ← (0 + 4 + 6) ÷ 3 = **3.33**

خوارزمية FCFS (F3)

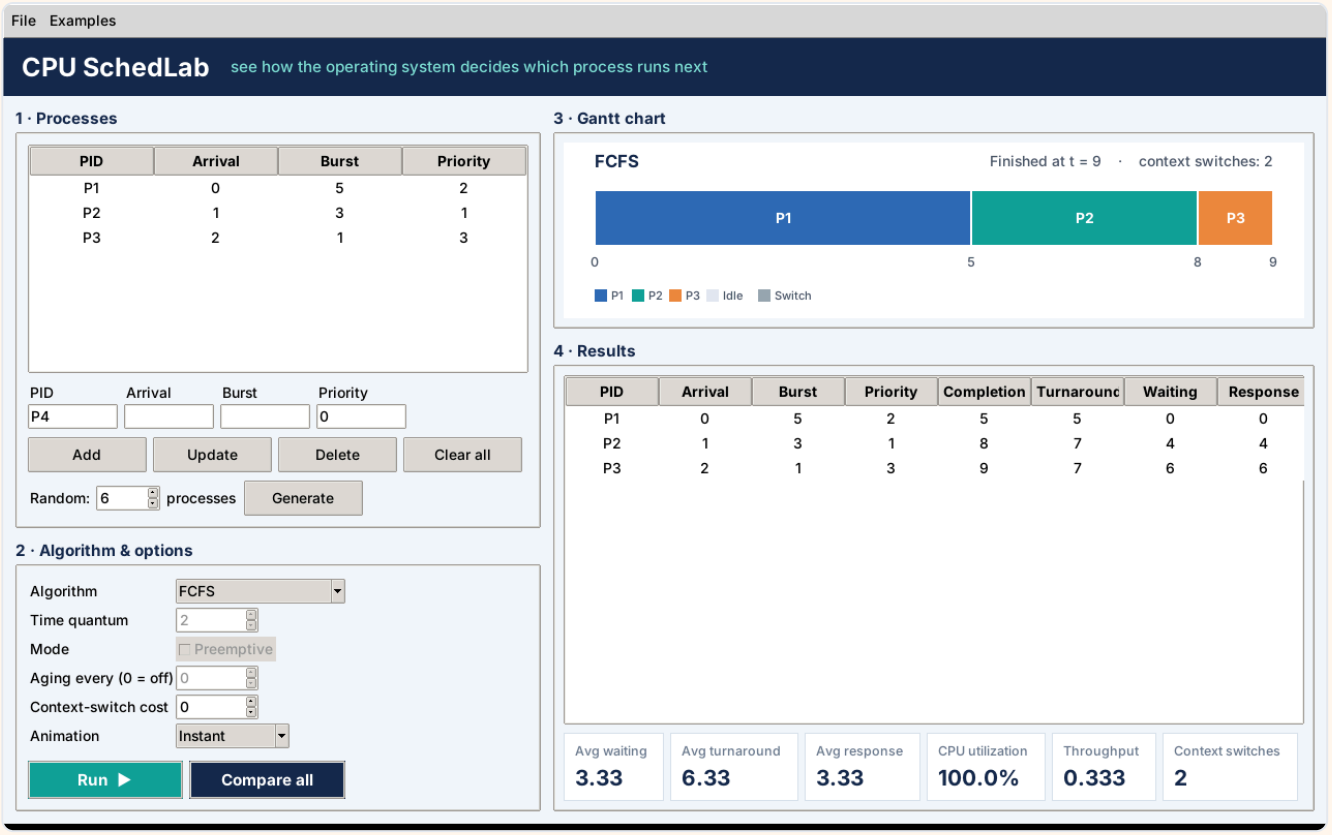
CPU_SchedLab.py — lines 102–113

```python
def fcfs_schedule(processes, options):
    """F3 FCFS: the first to ARRIVE runs first, until it finishes (non-preemptive)."""
    cpu = Dispatcher(options.switch_cost)
    time = 0
    for p in sorted(processes, key=lambda p: p.arrival):   # ready queue = FIFO by arrival
        if time < p.arrival:                                # nobody here yet -> CPU idle
            cpu.idle(time, p.arrival)
            time = p.arrival
        time = cpu.dispatch(p.pid, time)                    # give it the CPU
        cpu.run(p.pid, time, time + p.burst)                # it runs its WHOLE burst
        time += p.burst
    return cpu.result("FCFS", processes)
```

🎤 شو بتحكي للدكتورة

هاي خوارزمية **FCFS**، يعني اللي بيوصل أول بيشتغل أول. أول اشي برتّب الـ processes حسب وقت الوصول، وهاد هو الـ **ready queue** وبيمشي **FIFO**. بعدين بمرق عليهم وحدة وحدة: إذا المعالج فاضي والـ process لسّا ما وصلت، المعالج بضل **idle** لحد ما توصل. بعدها بعطيها المعالج عن طريق الـ dispatcher، وبتضلها شغّالة لحد ما تخلّص الـ burst كله، وما حدا بيقدر يقاطعها، عشان هيك FCFS هي **non-preemptive**. بمثالنا: P1 من 0 لـ 5، بعدين P2 لـ 8، بعدين P3 لـ 9، ومعدّل الانتظار طلع **3.33**. مشكلتها اسمها **Convoy effect**: إذا إجت process طويلة بالأول، كل الـ processes القصيرة بتضطر تستنّاها.

🖥️ شو بتعرض عالشاشة



بفتح البرنامج، بختار FCFS وبكبس **Run**. بيطلع الـ Gantt chart: P1 من 0 لـ 5، و P2 لـ 8، و P3 لـ 9، ومعدّل الانتظار **3.33**، نفس الرقم اللي بالـ proposal. وبعدين بجرّب أضيف process فيها burst = 0 عشان أورّيكي رسالة الخطأ.

💬 إذا سألتك الدكتورة

❓ ليش FCFS ممكن تكون سيئة؟

بسبب الـ Convoy effect: process طويلة بالأول بتخلّي كل القصار يستنّوا.

❓ إذا وصلت اثنتين بنفس الوقت؟

بتشتغل اللي انضافت أول، لأنه الترتيب ثابت (stable sort).

❓ شو علاقة الكلاس Process بالـ PCB؟

الـ PCB هو المكان اللي النظام بيحفظ فيه معلومات الـ process، وإحنا أخذنا منه اللي بيلزم الجدولة: الاسم والوصول والـ burst والـ priority.

Phase 3 · 17 ديسمبر الـ Random workload وتجميع كل الأجزاء

🧠 الفكرة بنظام التشغيل · Process creation

بنظام التشغيل الـ processes بتنخلق طول الوقت: process أب (parent) بتعمل process ابن (child)، بـ Linux عن طريق **fork()** وبويندوز **CreateProcess()**، وكل وحدة بيصيرلها PCB وبتفوت عالـ ready queue. التوليد العشوائي بمشروعنا بيحاكي هالشي: بيعمل processes جديدة بأوقات وصول وأطوال مختلفة، زي ما بيصير بالجهاز الحقيقي، عشان نجرّب الخوارزميات على حالات كتير.

توليد Processes عشوائية (F2)

CPU_SchedLab.py — lines 94–99

```
94  def random_workload(count=6, max_arrival=10, max_burst=10, max_priority=5):
95      """F2: generate `count` random processes P1..Pn for quick testing."""
96      if not 1 <= count <= 50:
97          raise ValueError("Number of processes must be between 1 and 50.")
98      return [Process(f"P{i}", random.randint(0, max_arrival), random.randint(1, max_burst),
99                      random.randint(0, max_priority)) for i in range(1, count + 1)]
```

🎤 شو بتحكي للدكتورة

هاي الدالة **random_workload**، بتولّد من process وحدة لـ 50 process بقيم عشوائية للوصول والـ burst والـ priority. هاي بتمثّل فكرة الـ **Process creation**، وبتفيدنا كتير بالتجريب، بدل ما نضل ندخّل الـ processes بإيدينا كل مرة.

الـ Core اللي بيجمّع شغل الفريق

CPU_SchedLab.py lines 342–360

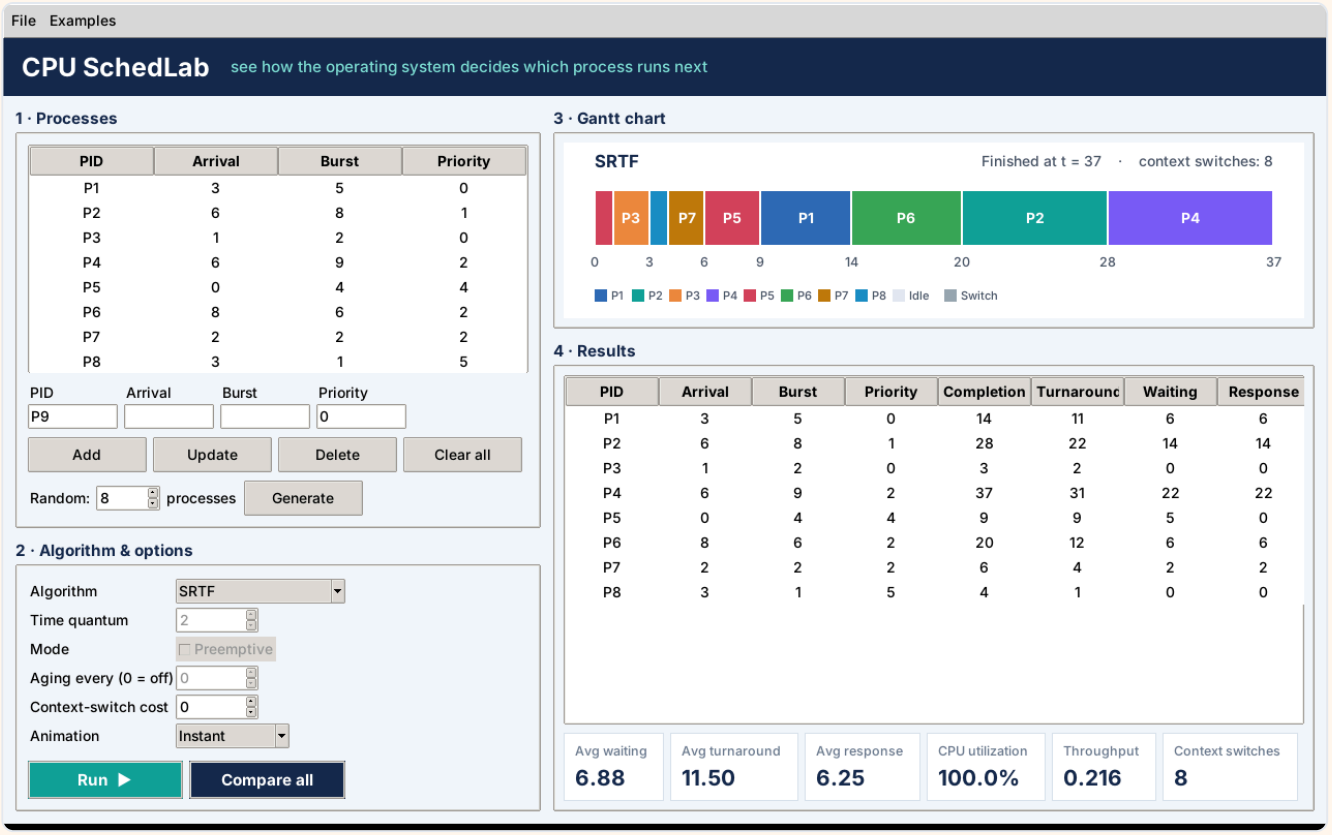
```python
ALGORITHMS = {
    "FCFS": fcfs_schedule,
    "SJF": sjf_schedule,
    "SRTF": srtf_schedule,
    "Round Robin": round_robin_schedule,
    "Priority": priority_schedule,
}


def run(algorithm, processes, options):
    """Check the input, pick the algorithm, run it on a COPY of the processes."""
    if not processes:
        raise ValueError("Add at least one process first.")
    ids = [p.pid for p in processes]
    if len(ids) != len(set(ids)):
        raise ValueError("Process ids must be unique.")
    if options.quantum <= 0 or options.aging < 0 or options.switch_cost < 0:
        raise ValueError("Quantum must be > 0; aging and switch cost cannot be negative.")
    return ALGORITHMS[algorithm](copy.deepcopy(processes), options)
```

🎤 شو بتحكي للدكتورة

هاد قلب البرنامج اللي بيجمع شغل الفريق كله. الـ **ALGORITHMS** هو قاموس بيربط اسم كل خوارزمية بالدالة تبعتها، وكل الدوال إلها نفس الشكل بالزبط: بتاخد الـ processes والخيارات وبترجع **Result** فيه الخط الزمني. والدالة **run** هي الوحيدة اللي بتناديها الواجهة: بتتأكد إنه في processes، وإنه ما في أسماء مكررة، وإنه الخيارات صح، وبعدين بتشغّل الخوارزمية على نسخة **deepcopy** من الـ processes عشان ما تخرب بيانات المستخدم. وبفضل هالتصميم، كل واحد بالفريق كتب وجرّب جزئه لحاله، وأنا جمّعت الأجزاء بسطر واحد لكل خوارزمية.

🖥️ شو بتعرض عالشاشة



بكبس **Generate**، بتطلع processes عشوائية، وبشغّل عليها أي خوارزمية عشان أوّري إنه كل الأجزاء شغّالة مع بعض.